

# VOXA STREAM

Auditoria técnica, refatoração estrutural e próximos passos

Versão auditada 0.7.4 → versão resultante 0.7.5

Windows x86-64 • Rust • Tauri 2 • React 19 • UDP nativo

Relatório emitido em 28 de setembro de 2026

# 1. Resultado executivo

APROVADO COM CORREÇÕES APLICADAS

O Voxa mantém a separação arquitetural exigida: o React/Tauri funciona como painel de controle, aprovação e diagnóstico; vídeo e áudio não passam pelo WebView. A mídia é capturada, convertida, codificada, cifrada, transportada, decodificada e apresentada por código nativo.

A auditoria encontrou quatro fragilidades concretas no caminho crítico: decisões de congestionamento espalhadas, tentativa sequencial de rota antes da seleção, feedback permissivo quanto ao tamanho e aceitação de encoders que não garantiam os controles de baixa latência. Todas foram corrigidas na versão 0.7.5.

O projeto passou por compilação Windows completa, Clippy sem avisos, 25 testes do núcleo UDP e 33 testes de painel, servidor, relay e implantação. As auditorias npm retornaram zero vulnerabilidades. O RustSec não encontrou vulnerabilidades, mas registrou sete avisos transitivos de manutenção/soundness ligados ao ecossistema Tauri.

| Área                  | Situação final       | Evidência principal                                                            |
|-----------------------|----------------------|--------------------------------------------------------------------------------|
| Isolamento do WebView | Conforme             | Sem HTML de mídia/WebRTC; CSP com media-src 'none'                             |
| Pipeline de GPU       | Conforme             | DXGI → D3D11 Video Processor → Media Foundation → swapchain D3D11              |
| UDP e FEC             | Conforme e reforçado | MTU 1.178 B; FEC apenas em IDR; descarte por atraso                            |
| Criptografia          | Conforme             | Argon2id + SPAKE2 P-256 + X25519 + ChaCha20-Poly1305                           |
| Dívida estrutural     | Parcial              | Novos módulos extraídos; transport.rs e native/mod.rs ainda devem ser fatiados |

## 2. Fronteira Tauri/Rust

### Violações de renderização D3D11 ou isolamento do Tauri

Nenhuma violação de mídia foi encontrada. Não existem tags <video>/<audio>, WebRTC, MediaStream ou captura do navegador. A janela de stream recebe frames por uma swapchain D3D11 vinculada ao HWND, fora do compositor de mídia HTML.

- O IPC transporta somente configuração, estado, aprovação, comandos de sessão e diagnóstico.
- A CSP bloqueia mídia web, frames e objetos: `media-src 'none'; frame-src 'none'; object-src 'none'`.
- As capabilities do Tauri estão limitadas a janela, atualização e processo.
- A permissão por esquema para HTTPS/WSS continua necessária porque o endpoint de matchmaking é configurável. Fixar origens no futuro reduzirá ainda mais a superfície.

### Fluxo de dados confirmado

| HOST                                                                                                        | REDE                                                                                                        | ESPECTADOR                                                                                        |
|-------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| DXGI Desktop Duplication<br>Textura D3D11 BGRA/HDR<br>↓<br>Video Processor NV12<br>↓<br>MF hardware encoder | Bitstream comprimido<br>↓<br>fragmentos $\leq 1.178$ B<br>↓<br>ChaCha20-Poly1305<br>↓<br>UDP direto ou VRLY | MF hardware decoder<br>Textura D3D11<br>↓<br>Video Processor<br>↓<br>swapchain HWND<br>Present(0) |

### Ponto corrigido

A chamada `SetMultithreadProtected(true)` existia, mas seu resultado era ignorado. Agora a captura e o espectador recusam iniciar caso o driver não confirme a proteção multithread D3D11, evitando raros deadlocks e corrupção de contexto.

## 3. Vídeo nativo e áudio

| Módulo       | Auditoria                                                    | Resultado                                                                   |
|--------------|--------------------------------------------------------------|-----------------------------------------------------------------------------|
| capture.rs   | Desktop Duplication, escolha de adaptador/saída, HDR, cursor | Textura D3D11 mantida na GPU; sem Map/Readback                              |
| converter.rs | Escala e BGRA/HDR → NV12                                     | D3D11 VideoProcessorBlt; recurso sem CPUAccessFlags                         |
| encoder.rs   | AV1 → H.265 → H.264, hardware, baixa latência                | ICodecAPI agora obrigatório; B=0, GOP curto, LowDelayVBR e bitrate dinâmico |
| decoder.rs   | MFT hardware e superfície DXGI                               | Saída ID3D11Texture2D; bitstream comprimido é a única cópia CPU necessária  |
| presenter.rs | Swapchain nativa, resize, letterbox, fullscreen              | Apresentação fora do WebView; recriação após device lost                    |
| audio.rs     | WASAPI e Opus                                                | 48 kHz estéreo; loopback geral ou processo; fila limitada                   |

### O que zero-copy significa aqui

Os pixels brutos não voltam para a RAM: a textura DXGI alimenta o conversor D3D11, a textura NV12 alimenta o encoder por `MFCreatDXGISurfaceBuffer`, e o decoder entrega outra textura DXGI ao presenter. O bitstream H.264/H.265/AV1 precisa existir na memória do processo para cifragem e packetização; isso é esperado e não viola o objetivo de zero-copy dos frames crus.

### Áudio isolado

O modo por processo usa `AUDIOCLIENT_ACTIVATION_TYPE_PROCESS_LOOPBACK` e inclui a árvore do processo selecionado. Esse caminho exige Windows 10 build 20348 ou superior. Em versões anteriores, o modo geral captura a saída do sistema e pode incluir Discord; a interface informa essa limitação. O Opus usa 48 kHz, dois canais e 20 ms por pacote, com bitrate adaptado à faixa de vídeo.

## 4. Transporte UDP e congestionamento

### Limites confirmados

| Regra            | Valor                  | Comportamento                                   |
|------------------|------------------------|-------------------------------------------------|
| Datagrama VOXA   | 1.178 bytes            | Reserva 22 bytes para VRLY sob MTU de 1.200     |
| Payload máximo   | 1.120 bytes            | Inclui margem para cabeçalho e tag AEAD         |
| Framentos/frame  | 4.096                  | Frame máximo de 4.587.520 bytes (~4,38 MiB)     |
| FEC              | XOR / 8 framentos      | Somente keyframes; recupera uma perda por grupo |
| Frames pendentes | 3                      | Limita memória sob tráfego hostil               |
| Pedido de IDR    | ≥ 250 ms entre pedidos | Máximo de quatro pedidos por segundo            |

### Código refatorado — decisão de congestionamento

```
pub struct CongestionDecision {
    pub target_bitrate_bps: u32,
    pub condition: NetworkCondition,
    pub max_delta_age: Duration,
    pub request_keyframe: bool,
}

pub fn update(&mut self, loss_pct: f32, rtt_ms: u32) -> CongestionDecision {
    let next = classify(loss_pct, rtt_ms);
    self.adjust_bitrate(next); // 0,8 a 35 Mbps
    CongestionDecision {
        target_bitrate_bps: self.bitrate_bps,
        condition: next,
        max_delta_age: match next {
            Healthy => 250.ms(), Constrained => 140.ms(), Critical => 80.ms(),
        },
        request_keyframe: recovered_after_sustained_critical,
    }
}
```

A implementação real permanece em Rust estável e usa `Duration`. O transmissor consulta a idade do delta antes de fragmentá-lo e também interrompe um delta quando já existe frame mais novo. Keyframes nunca são descartados pela regra de idade; recebem pacing e FEC.

A rota desconhecida agora usa `tokio::join!` para tentar endereço direto e relay simultaneamente. O primeiro pacote autenticado aceito pelo antirreplay fixa a rota; perda de autenticação reseta a seleção.

## 5. Segurança e modelo de confiança

| Camada          | Mecanismo                  | Verificação                                              |
|-----------------|----------------------------|----------------------------------------------------------|
| Senha da sala   | Argon2id                   | 64 MiB, 3 iterações, paralelismo 1, saída 32 B           |
| PAKE            | SPAKE2 P-256 RFC 9382      | Shares efêmeros + confirmação mútua; senha não sai do PC |
| Acordo de chave | X25519                     | Segredo distinto por espectador e por sessão             |
| Vínculo         | SHA-256                    | Liga X25519 ao resultado autenticado do SPAKE2           |
| Datagrama       | ChaCha20-Poly1305          | Confidencialidade, integridade e autenticação            |
| Replay          | Janela por stream/sequence | Duplicatas e pacotes antigos são recusados               |
| Relay           | VRLY autenticado           | Vê IP/volume, mas nunca abre o datagrama VOXA            |

### Servidor de matchmaking

O Node/Socket.IO valida identificadores, limita conexões e requisições, exige hello em prazo curto, limita quatro espectadores por host e encaminha apenas shares/confirmations públicas do PAKE e candidatos de rota. Não recebe senha, prova reutilizável, chave de mídia ou mídia.

### Defesa adicional aplicada

O parser de feedback agora aceita somente 8 bytes do formato legado ou 20 bytes do formato atual. Antes, qualquer carga com pelo menos 8 bytes era aceita e bytes excedentes eram ignorados. A mudança elimina ambiguidades de protocolo e facilita fuzzing determinístico.

## 6. Refatoração estrutural

### O que foi extraído agora

- `transport/feedback.rs`: serialização, validação e compatibilidade do feedback.
- `transport/packetizer.rs`: cálculo de fragmentos e paridade XOR.
- `congestion.rs`: decisão completa e limitador de keyframe testável.
- `route.rs`: disputa paralela direta/relay.

### Plano para fatiar `transport.rs`

| Etapa | Novo módulo                                            | Responsabilidade                                          |
|-------|--------------------------------------------------------|-----------------------------------------------------------|
| 1     | <code>session.rs</code>                                | Construção do estado, spawn, stop e ownership das tarefas |
| 2     | <code>receiver.rs</code>                               | Open AEAD, antirreplay, remontagem, expiração e despacho  |
| 3     | <code>senders/video.rs</code>                          | Config, descarte por idade, pacing, fragmentação e FEC    |
| 4     | <code>senders/audio.rs</code> / <code>cursor.rs</code> | Filas limitadas e envio prioritário independente          |
| 5     | <code>heartbeat.rs</code>                              | Ping/pong, relógio, feedback, rota e congestionamento     |
| 6     | <code>state.rs</code>                                  | Handles, WakeSignal e DatagramHub                         |
| 7     | <code>wire.rs</code>                                   | Seal, Meta, sequence e timestamps                         |

### Plano para `native/mod.rs`

Manter `native/mod.rs` como fachada compatível com os comandos Tauri e mover a implementação, em commits independentes, para `commands/session.rs`, `commands/security.rs`, `commands/devices.rs`, `peer.rs` e `diagnostics/export.rs`. Essa ordem reduz conflitos e permite compilar cada extração antes da próxima.

## 7. Qualidade, testes e cadeia de fornecimento

| Verificação         | Resultado                    | Observação                                              |
|---------------------|------------------------------|---------------------------------------------------------|
| Cargo check Windows | Passou                       | Backend completo e APIs Win32                           |
| Clippy -D warnings  | Passou                       | Todos os alvos compiláveis                              |
| Núcleo Rust         | 25/25                        | Cripto, replay, MTU, FEC, jitter, perda e remontagem    |
| Node/TypeScript     | 33/33                        | Painel, rate limits, PAKE, relay, implantação e release |
| npm audit           | 0 vulnerabilidades           | Raiz e servidor                                         |
| RustSec             | 0 vulnerabilidades; 7 avisos | Transitivos do Tauri; monitorados                       |
| Fuzz CI             | Configurado                  | Protocol e reassembly, execução semanal e manual        |
| Dependabot          | Configurado                  | npm raiz/servidor, Cargo raiz/core/fuzz e Actions       |

### Limitação da validação local

O executável principal compilou. O binário de testes Tauri com toolchain GNU requer localizar `WebView2Loader.dll` durante o link; o script `scripts/test-rust.mjs` prepara esse caminho e o job Windows do GitHub Actions é a verificação definitiva. Os testes do núcleo independente foram executados localmente e passaram.

## 8. Melhorias recomendadas

| Prioridade | Melhoria                                          | Benefício                                                                |
|------------|---------------------------------------------------|--------------------------------------------------------------------------|
| P0         | Teste real em dois PCs e CGNAT                    | Valida drivers, firewall, áudio e relay que mocks não reproduzem         |
| P1         | Timestamps monotônicos no fio com epoch de sessão | Evita efeitos de ajuste do relógio do Windows na latência captura→tela   |
| P1         | Telemetria por etapa                              | Separar captura, encode, rede, decode e present em P50/P95/P99           |
| P1         | FEC adaptativo                                    | Alterar grupo/redundância de IDR conforme perda sem onerar rede saudável |
| P1         | Segundo relay gratuito em região distinta         | Resiliência a indisponibilidade e menor RTT fora de São Paulo            |
| P2         | HDR10 completo com metadata                       | Preserva cor/brilho em monitores e jogos HDR                             |
| P2         | Controle remoto com consentimento forte           | Mouse/gamepad somente após autorização visível e revogável               |
| P2         | Teste HIL de GPU                                  | Matriz AMD/NVIDIA/Intel com captura, encoder, decoder e device lost      |

### O que eu adicionaria ao código

- Um relógio monotônico de sessão compartilhado por áudio e vídeo, mantendo wall-clock apenas para diagnóstico.
- Uma máquina de estados formal para a sessão: idle → authenticating → awaiting-approval → connecting → streaming → recovering → closed.
- Propriedades/testes gerados para parsers de wire protocol além do fuzzing, cobrindo invariantes de tamanho e monotonicidade.
- Um perfil de compatibilidade registrado por GPU/driver, para desabilitar automaticamente controles de codec conhecidos como quebrados.
- Um teste de soak de duas horas com perda/jitter variável e verificação de memória, handles D3D/COM e filas.

## 9. O que você precisa fazer

### Ações imediatas

| Ordem | Ação                                   | Critério de sucesso                                       |
|-------|----------------------------------------|-----------------------------------------------------------|
| 1     | Instalar 0.7.5 em dois computadores    | Ambos mostram a mesma versão e codecs por GPU             |
| 2     | Começar com H.264 e 1080p60            | Aprovação aparece; imagem e áudio iniciam sem tela branca |
| 3     | Testar áudio geral e por processo      | Jogo é ouvido; Discord/Voxa não retorna no modo isolado   |
| 4     | Testar direta e relay                  | Diagnóstico identifica a rota; CGNAT conecta via relay    |
| 5     | Oscilar Wi-Fi e suspender/retomar      | Sessão recupera ou pede reentrada sem crash               |
| 6     | Trocar monitor e fullscreen            | Resize/letterbox corretos e captura continua              |
| 7     | Exportar diagnóstico em qualquer falha | Arquivo contém GPU, driver, codec, rota, perda e erro     |

### Não é necessário configurar Supabase

O Voxa não depende mais de Supabase. O serviço necessário é o matchmaking HTTPS/WSS e o relay UDP. Mantenha a VM Oracle dentro dos recursos Always Free e a porta 3479/UDP liberada somente para o relay; não exponha portas administrativas além das necessárias.

### Sinal para interromper distribuição

Se nenhum codec aparecer, se o driver recusar ICodecAPI/baixa latência, se houver tela branca, ou se a rota ficar em recovering por mais de alguns segundos, não distribua o instalador dessa máquina como validado. Exporte o diagnóstico e use o erro detalhado para identificar GPU/driver/controle ausente.

## 10. Conclusão

A versão 0.7.5 reforça o caminho que mais influencia fluidez: remove trabalho de frames que já perderam valor, reage de modo explícito à pressão da rede, solicita IDR no momento de recuperação, disputa NAT direto e relay sem atraso sequencial e recusa encoders que não conseguem cumprir o contrato adaptativo.

A arquitetura central está correta para um streamer nativo: controle no Tauri, pixels no D3D11, codecs no Media Foundation, áudio no WASAPI/Opus e mídia cifrada em UDP. O principal risco restante não é uma violação estrutural; é a variabilidade real de drivers, GPUs e redes domésticas, que exige a matriz de testes em hardware descrita neste relatório.

### ENTREGA TÉCNICA

- Código promovido para 0.7.5.
- Controle de congestionamento refatorado e testado.
- Feedback e packetização/FEC modularizados.
- Corrida direta/relay paralela.
- Contrato de encoder e proteção D3D11 reforçados.
- Plano de decomposição de transport.rs e native/mod.rs documentado.
- PDF e relatório Markdown incluídos no repositório.

**Parecer: pronto para build/release de validação, condicionado ao teste prático em dois computadores antes de declarar estabilidade em produção.**